Transport Interface Reference

Wire protocol and message schemas for Cross Guard message transports

Overview

Cross Guard supports four transport providers for relaying messages between Mattermost instances: NATS, Azure Queue Storage, Azure Blob Storage (batched WAL), and Azure Service Bus. Each provider has different delivery guarantees, latency characteristics, and infrastructure requirements. Text events (posts, edits, deletes, reactions) are wrapped in a message envelope (serialized as JSON or XML, per connection configuration) and delivered via the configured transport. File attachments use a provider-specific subsystem: NATS JetStream Object Store, Azure Blob Storage (paired sidecar for Azure Queue and Azure Service Bus, or the same container for Azure Blob), depending on the transport.

Transport Providers

Aspect	NATS	Azure Queue Storage	Azure Blob Storage	Azure Service Bus
Delivery Model	Pub/sub (push)	Queue polling (pull, 5s interval)	Batched WAL blob (60s flush, lease-based read)	PeekLock pull (32-msg batch, 5s max wait)
Delivery Guarantee	At-most-once	At-least-once	At-least-once	At-least-once (PeekLock with server-side DeliveryCount)
Message Size Limit	1 MB (NATS server default, not plugin-enforced; no automatic truncation)	48,000 bytes (plugin-enforced truncation; before Base64 to 64 KB wire limit)	Bounded by batch blob size (many messages per blob)	192,000 bytes default; 256 KiB Standard tier ceiling, 100 MiB on Premium

Aspect	NATS	Azure Queue Storage	Azure Blob Storage	Azure Service Bus
File Transfer	JetStream Object Store	Azure Blob Storage (15s poll interval)	Same container as WAL (deferred file blobs)	Azure Blob Storage sidecar (15s poll, independent credentials)
Encryption	TLS (optional)	TLS always (Azure endpoint)	TLS always (Azure endpoint)	TLS always (AMQP over TLS)
Authentication	Token, credentials, or mTLS	Shared key, or Service Principal (Azure AD client secret)	Shared key, or Service Principal (Azure AD client secret)	SAS connection string, or Service Principal (Azure AD client secret)
Message Encoding	Raw bytes	Base64 over wire	Length-prefixed batch within the blob	AMQP message body (raw bytes)
Auto-Provisioning	No (server must exist)	Yes in shared-key mode (queue and container auto-created); no in Service Principal mode (pre-provision required)	Yes in shared-key mode (container auto-created); no in Service Principal mode (pre-provision required)	No (queue must be pre-created with chosen LockDuration and MaxDeliveryCount)

NATS Transport Summary

Text Transport	NATS core publish/subscribe
File Transport	NATS JetStream Object Store (bucket: `crossguard-files`)
Wire Format	JSON or XML (UTF-8 encoded bytes, per connection config) for text; binary for files

Subject Prefix	`crossguard.` (required for all subjects)
Subject Pattern	`crossguard.{domain}`, e.g. `crossguard.high`, `crossguard.low`
Retry Policy	Up to 3 attempts with exponential backoff (500ms base, 5s max) for text relay
Text Concurrency	Up to 256 concurrent text relay operations per plugin instance
File Concurrency	Up to 32 concurrent file upload/download operations per plugin instance

Data Flow (NATS)

Outbound (Local to Remote)

```
User action in Mattermost
    |
    v
Plugin hook fires (MessageHasBeenPosted, etc.)
    |
    v
Check: channel relay enabled? team linked to outbound connection?
    |
    v
Build Envelope (type + payload), serialize per-connection format (JSON or XML)
    |
    v
Queue for async publish (semaphore-gated, max 256)
    |
    v
Publish to NATS subject with retry (3x, exponential backoff)
    |
    v
Flush to confirm delivery
```

Inbound (Remote to Local)

```
NATS message arrives on subscribed subject
    |
    v
```

```
Acquire relay semaphore (max 256, non-blocking; dropped if full)
    |
    v
Auto-detect format (JSON or XML) and deserialize Envelope
    |
    v
Dispatch by message type (post, update, delete, reaction, test)
    |
    v
Resolve team and channel by name (apply rewrite rules if configured)
    |
    v
Verify team AND channel are linked to this inbound connection
    |
    v
Resolve user (lookup by username or use relay bot)
    |
    v
Create/Update/Delete post or reaction in Mattermost
    |
    v
Store post ID mapping (remote ID to local ID)
    |
    v
Set "crossguard_relayed" property (prevents relay loops)
```

Message Envelope

Every message sent via the transport layer uses this standard envelope. The `type` field identifies the message kind. Exactly one payload field is present per message. The envelope can be serialized as JSON or XML depending on the outbound connection's `message_format` setting. Inbound messages are auto-detected by inspecting the first non-whitespace byte.

NATS **MessageEnvelope**

Schema

`type`	**string, required**. One of the message type values. Determines which payload field is present.

`timestamp`	**string, required**. UTC timestamp in RFC 3339 format (e.g. `2026-03-27T14:30:00Z`). Set when the envelope is created.
`post_message`	**object, optional**. Present when type is `crossguard_post` or `crossguard_update`. See PostMessage.
`delete_message`	**object, optional**. Present when type is `crossguard_delete`. See DeleteMessage.
`reaction_message`	**object, optional**. Present when type is `crossguard_reaction_add` or `crossguard_reaction_remove`. See ReactionMessage.
`test_message`	**object, optional**. Present when type is `crossguard_test`. See TestMessage.

JSON Example

```
{
  "type": "crossguard_post",
  "timestamp": "2026-03-27T14:30:00Z",
  "post_message": {
    "post_id": "8swgtrrdiff89yj1z33pofo15r",
    "channel_id": "9a1f4h6e17yfmk9w6pxs3jqd3e",
    "channel_name": "town-square",
    "team_id": "5f3h8k2m41ygdj7w8pxr6nqb9c",
    "team_name": "test-a",
    "user_id": "7m4k9j2p63ynfr8x1qwt5hab4e",
    "username": "alice",
    "message": "Hello from the other side",
    "create_at": 1711547400000
  }
}
```

XML Example

Note: Actual wire output is compact (no indentation). Formatted here for readability.

```
<?xml version="1.0" encoding="UTF-8"?>
<CrossguardMessage xmlns="urn:mattermost-crossguard">
  <Type>crossguard_post</Type>
  <Timestamp>2026-03-27T14:30:00Z</Timestamp>
  <PostMessage>
    <PostID>8swgtrrdiff89yj1z33pofo15r</PostID>
    <ChannelID>9a1f4h6e17yfmk9w6pxs3jqd3e</ChannelID>
```

```
    <ChannelName>town-square</ChannelName>
    <TeamID>5f3h8k2m41ygdj7w8pxr6nqb9c</TeamID>
    <TeamName>test-a</TeamName>
    <UserID>7m4k9j2p63ynfr8x1qwt5hab4e</UserID>
    <Username>alice</Username>
    <MessageText>Hello from the other side</MessageText>
    <CreateAt>1711547400000</CreateAt>
  </PostMessage>
</CrossguardMessage>
```

Message Types

The `type` field in the envelope acts as a discriminator. Each value maps to a specific payload field in the envelope.

Type Value	Go Constant	Payload Schema	Descrip
`crossguard_post`	`MessageTypePost`	PostMessage	A new p was crea
`crossguard_update`	`MessageTypeUpdate`	PostMessage	An existi post was edited
`crossguard_delete`	`MessageTypeDelete`	DeleteMessage	A post w deleted
`crossguard_reaction_add`	`MessageTypeReactionAdd`	ReactionMessage	A reacti was add to a pos
`crossguard_reaction_remove`	`MessageTypeReactionRemove`	ReactionMessage	A reacti was removed from a p

Type Value	Go Constant	Payload Schema	Descrip
`crossguard_test`	`MessageTypeTest`	TestMessage	Connect test message

PostMessage

Used for both `crossguard_post` (new posts) and `crossguard_update` (edits). For updates, the fields reflect the post state after the edit.

PAYLOAD **PostMessage**

Fields

Field	Type	Required	Description
`post_id`	string	Yes	Unique identifier of the post on the originating server
`root_id`	string	No	ID of the root post if this is a threaded reply. Omitted for top-level posts.
`channel_id`	string	Yes	ID of the channel on the originating server
`channel_name`	string	Yes	URL-safe channel name (used for cross-domain resolution)
`team_id`	string	Yes	ID of the team on the originating server
`team_name`	string	Yes	URL-safe team name (used for cross-domain resolution, subject to rewrite rules)
`user_id`	string	Yes	ID of the post author on the originating server
`username`	string	Yes	Username of the post author
`message`	string	Yes	Markdown-formatted message body

Field	Type	Required	Description
`create_at`	integer (int64)	Yes	Unix timestamp in milliseconds when the post was originally created

Example: New Post

```
{
  "post_id": "8swgtrrdiff89yj1z33pofo15r",
  "channel_id": "9a1f4h6e17yfmk9w6pxs3jqd3e",
  "channel_name": "town-square",
  "team_id": "5f3h8k2m41ygdj7w8pxr6nqb9c",
  "team_name": "test-a",
  "user_id": "7m4k9j2p63ynfr8x1qwt5hab4e",
  "username": "alice",
  "message": "Hello from the other side",
  "create_at": 1711547400000
}
```

Example: Threaded Reply

```
{
  "post_id": "kf7g2m4x91yrnh5w3pqs8jab6e",
  "root_id": "8swgtrrdiff89yj1z33pofo15r",
  "channel_id": "9a1f4h6e17yfmk9w6pxs3jqd3e",
  "channel_name": "town-square",
  "team_id": "5f3h8k2m41ygdj7w8pxr6nqb9c",
  "team_name": "test-a",
  "user_id": "3n6k8j1p42ygfr9x7qwt5hab2c",
  "username": "bob",
  "message": "Replying in thread",
  "create_at": 1711547460000
}
```

DeleteMessage

Identifies a post that was deleted on the originating server. The receiving instance uses the `post_id` to look up the local copy via its post ID mapping and delete it.

PAYLOAD **DeleteMessage**

Fields

Field	Type	Required	Description
`post_id`	string	Yes	Unique identifier of the deleted post on the originating server
`channel_id`	string	Yes	ID of the channel the post belonged to
`channel_name`	string	Yes	URL-safe channel name
`team_id`	string	Yes	ID of the team on the originating server
`team_name`	string	Yes	URL-safe team name

Example

```
{
  "post_id": "8swgtrrdiff89yj1z33pofo15r",
  "channel_id": "9a1f4h6e17yfmk9w6pxs3jqd3e",
  "channel_name": "town-square",
  "team_id": "5f3h8k2m41ygdj7w8pxr6nqb9c",
  "team_name": "test-a"
}
```

ReactionMessage

Represents a reaction added to or removed from a post. Used by both `crossguard_reaction_add` and `crossguard_reaction_remove`. The receiving instance resolves the local post via its post ID mapping and applies or removes the reaction.

PAYLOAD **ReactionMessage**

Fields

Field	Type	Required	Description
`post_id`	string	Yes	ID of the post the reaction applies to
`channel_id`	string	Yes	ID of the channel containing the post
`channel_name`	string	Yes	URL-safe channel name
`team_id`	string	Yes	ID of the team on the originating server
`team_name`	string	Yes	URL-safe team name
`user_id`	string	Yes	ID of the user who added or removed the reaction
`username`	string	Yes	Username of the user
`emoji_name`	string	Yes	Emoji name without colons (e.g. `thumbsup`, not `:thumbsup:`)

Example

```
{
  "post_id": "8swgtrrdiff89yj1z33pofo15r",
  "channel_id": "9a1f4h6e17yfmk9w6pxs3jqd3e",
  "channel_name": "town-square",
  "team_id": "5f3h8k2m41ygdj7w8pxr6nqb9c",
  "team_name": "test-a",
  "user_id": "7m4k9j2p63ynfr8x1qwt5hab4e",
  "username": "alice",
  "emoji_name": "thumbsup"
}
```

TestMessage

A lightweight message used to verify transport connectivity. Sent via the `POST /api/v1/test-connection` REST endpoint when testing an outbound connection. The receiving instance logs receipt and confirms the connection is functional.

PAYLOAD **TestMessage**

Fields

Field	Type	Required	Description
`id`	string	Yes	A Mattermost-generated unique identifier for the test request

Example

```
{
  "id": "dj7f8k2m41ygsr9w6pxt3nqb4e"
}
```

Complete Wire Format Examples

Each example below shows the full JSON envelope as it appears on the wire. For XML equivalents, see the XML Format section and the `schema/crossguard.xsd` schema definition.

New Post

```
{
  "type": "crossguard_post",
  "timestamp": "2026-03-27T14:30:00Z",
  "post_message": {
    "post_id": "8swgtrrdiff89yj1z33pofo15r",
    "channel_id": "9a1f4h6e17yfmk9w6pxs3jqd3e",
    "channel_name": "town-square",
    "team_id": "5f3h8k2m41ygdj7w8pxr6nqb9c",
    "team_name": "test-a",
    "user_id": "7m4k9j2p63ynfr8x1qwt5hab4e",
    "username": "alice",
```

```
    "message": "Hello from the other side",
    "create_at": 1711547400000
  }
}
```

Post Edit

```
{
  "type": "crossguard_update",
  "timestamp": "2026-03-27T14:31:00Z",
  "post_message": {
    "post_id": "8swgtrrdiff89yj1z33pofo15r",
    "channel_id": "9a1f4h6e17yfmk9w6pxs3jqd3e",
    "channel_name": "town-square",
    "team_id": "5f3h8k2m41ygdj7w8pxr6nqb9c",
    "team_name": "test-a",
    "user_id": "7m4k9j2p63ynfr8x1qwt5hab4e",
    "username": "alice",
    "message": "Hello from the other side (edited)",
    "create_at": 1711547400000
  }
}
```

Post Delete

```
{
  "type": "crossguard_delete",
  "timestamp": "2026-03-27T14:32:00Z",
  "delete_message": {
    "post_id": "8swgtrrdiff89yj1z33pofo15r",
    "channel_id": "9a1f4h6e17yfmk9w6pxs3jqd3e",
    "channel_name": "town-square",
    "team_id": "5f3h8k2m41ygdj7w8pxr6nqb9c",
    "team_name": "test-a"
  }
}
```

Reaction Added

```
{
  "type": "crossguard_reaction_add",
  "timestamp": "2026-03-27T14:33:00Z",
  "reaction_message": {
    "post_id": "8swgtrrdiff89yj1z33pofo15r",
    "channel_id": "9a1f4h6e17yfmk9w6pxs3jqd3e",
    "channel_name": "town-square",
    "team_id": "5f3h8k2m41ygdj7w8pxr6nqb9c",
    "team_name": "test-a",
    "user_id": "3n6k8j1p42ygfr9x7qwt5hab2c",
    "username": "bob",
    "emoji_name": "thumbsup"
  }
}
```


Reaction Removed


```
{
  "type": "crossguard_reaction_remove",
  "timestamp": "2026-03-27T14:34:00Z",
  "reaction_message": {
    "post_id": "8swgtrrdiff89yj1z33pofo15r",
    "channel_id": "9a1f4h6e17yfmk9w6pxs3jqd3e",
    "channel_name": "town-square",
    "team_id": "5f3h8k2m41ygdj7w8pxr6nqb9c",
    "team_name": "test-a",
    "user_id": "3n6k8j1p42ygfr9x7qwt5hab2c",
    "username": "bob",
    "emoji_name": "thumbsup"
  }
}
```


Connectivity Test


```
{
  "type": "crossguard_test",
  "timestamp": "2026-03-27T14:35:00Z",
  "test_message": {
    "id": "dj7f8k2m41ygsr9w6pxt3nqb4e"
  }
}
```

Connection Configuration (NATS)

Each NATS connection (inbound or outbound) is configured in the System Console. This schema is stored in plugin settings and is not transmitted over the wire. It is included here for reference when setting up NATS integrations. The `provider` field selects the transport backend. NATS-specific fields (`address` , `subject` , `tls_enabled` , etc.) are nested under the `"nats"` key in JSON.

CONFIG **NATSConnection**

Fields

Field	Type	Required	Description
Connection Fields (top-level in JSON)			
`name`	string	Yes	Connection identifier. Must start and end with a lowercase letter or number. Hyphens allowed between segments, no consecutive hyphens (pattern: `^[a-z0-9]+(-[a-z0-9]+)*$`).
`provider`	string	No	Transport provider: `"nats"` , `"azure-queue"` , `"azure-blob"` , or `"azure-servicebus"` . Default: `"nats"` .
`file_transfer_enabled`	boolean	No	Enable file attachment relay via JetStream Object Store. Default: `false` .
`file_filter_mode`	string	No	File extension filter: `""` (no filter), `"allow"` (whitelist), or `"deny"` (blacklist). Default: `""` .
`file_filter_types`	string	Conditional	Comma-separated extensions (e.g. `.pdf,.docx,.png`). Case-insensitive. Required when

Field	Type	Required	Description
			`file_filter_mode` is `"allow"` or `"deny"`.
`message_format`	string	No	Wire format for outbound messages: `"json"` (default) or `"xml"`. Use XML when sending through a Cross Domain Solution (CDS). Inbound messages are auto-detected regardless of this setting.
NATS Provider Fields (nested under `"nats"` key in JSON)			
`address`	string	Yes	NATS server address (e.g. `nats://nats:4222`).
`subject`	string	Yes	NATS subject. Must start with `crossguard.` (e.g. `crossguard.high`).
`tls_enabled`	boolean	No	Enable TLS encryption. Default: `false`.
`auth_type`	string	No	Authentication method: `none`, `token`, or `credentials`. Default: `none`.
`token`	string	Conditional	Bearer token. Required when `auth_type` is `token`.
`username`	string	Conditional	Username. Required when `auth_type` is `credentials`.
`password`	string	Conditional	Password. Required when `auth_type` is `credentials`.
`client_cert`	string	No	Filesystem path to the client TLS certificate.
`client_key`	string	No	Filesystem path to the client TLS private key.

Field	Type	Required	Description
`ca_cert`	string	No	Filesystem path to the CA certificate for server verification.

Example: Token Authentication

```
{
  "name": "high-side",
  "provider": "nats",
  "nats": {
    "address": "nats://nats:4222",
    "subject": "crossguard.high",
    "tls_enabled": false,
    "auth_type": "token",
    "token": "my-secret-token"
  }
}
```

Example: TLS with Credentials

```
{
  "name": "secure-relay",
  "provider": "nats",
  "nats": {
    "address": "nats://nats.example.com:4222",
    "subject": "crossguard.relay",
    "tls_enabled": true,
    "auth_type": "credentials",
    "username": "crossguard-svc",
    "password": "s3cret",
    "client_cert": "/etc/crossguard/client.crt",
    "client_key": "/etc/crossguard/client.key",
    "ca_cert": "/etc/crossguard/ca.crt"
  }
}
```

Example: File Transfer with Allow Filter

```
{
  "name": "high-side",
  "provider": "nats",
```

```
  "file_transfer_enabled": true,
  "file_filter_mode": "allow",
  "file_filter_types": ".pdf,.docx,.png,.jpg",
  "nats": {
    "address": "nats://nats:4222",
    "subject": "crossguard.high",
    "auth_type": "token",
    "token": "my-secret-token"
  }
}
```

Example: XML Format for CDS

```
{
  "name": "cds-relay",
  "provider": "nats",
  "message_format": "xml",
  "nats": {
    "address": "nats://nats:4222",
    "subject": "crossguard.cds",
    "tls_enabled": true,
    "auth_type": "token",
    "token": "my-secret-token"
  }
}
```

Connection Behavior (NATS)

Parameter	Value	Description
Connect Timeout (test)	30 seconds	One-time connections used by the test-connection API endpoint
Connect Timeout (relay)	5 seconds	Persistent relay connections established at plugin activation
Max Reconnects	Unlimited	Relay connections attempt reconnection indefinitely
Reconnect Wait	2 seconds	Delay between reconnection attempts

Parameter	Value	Description
Publish Retries	3	Maximum attempts per message publish
Publish Base Delay	500ms	Initial backoff delay between retries
Publish Max Delay	5 seconds	Maximum backoff delay between retries
Text Relay Semaphore	256	Maximum concurrent text relay operations (semaphore-gated)
File Semaphore	32	Maximum concurrent file upload/download operations (separate from text relay)
Object Store TTL	1 hour	JetStream Object Store bucket TTL for file objects
File Post Mapping Retry	3 attempts	Inbound file watcher retries post mapping lookup with linear backoff (1s, 2s)

File Transfer Protocol (NATS JetStream)

File attachment relay uses NATS JetStream Object Store as an out-of-band binary transfer mechanism. Text and file relay are completely independent systems that share the same NATS connection but use different protocols. File operations cannot starve text relay because they use a separate semaphore.

Object Store Configuration

Parameter	Value	Description
Bucket Name	`crossguard-files`	Auto-created on first use via `CreateOrUpdateObjectStore`
TTL	1 hour	Objects are automatically purged after TTL. No receiver-side deletion (supports fan-out to multiple receivers).
Object Key	`{postID}/{randomID}`	Post ID from the originating server combined with a generated unique identifier. Avoids filename encoding issues.

Parameter	Value	Description
Max File Size	Server's `MaxFileSize`	Governed by `FileSettings.MaxFileSize` in the Mattermost server config. Default: 100 MB.

Object Metadata Headers

Each uploaded object carries three custom NATS headers that the receiving instance uses to route and attach the file to the correct local post.

Header	Type	Description
`X-Post-Id`	string	The originating server's post ID. Used to look up the local post via the post ID mapping (`pm-{connName}-{remotePostID}`).
`X-Conn-Name`	string	The connection name that uploaded the file. Receivers filter objects by this header to process only files from the matching inbound connection.
`X-Filename`	string	Original filename of the attachment as it appeared on the sending server.

Outbound File Flow

```
User posts message with file attachment
    |
    v
Plugin hook fires (MessageHasBeenPosted)
    |
    +--> Text relay (core NATS pub/sub, unchanged)
    |
    +--> File relay (for each enabled outbound connection):
         |
         v
         Check: file_transfer_enabled? File passes filter? Under size limit?
             |
             v
         Download file bytes from Mattermost (once, shared across connections)
             |
             v
         Acquire file semaphore (max 32, non-blocking; skipped if full)
             |
```

```
        v
    Get or create Object Store bucket ("crossguard-files")
        |
        v
    Upload file bytes to Object Store
    Key: {postID}/{randomID}
    Headers: X-Post-Id, X-Conn-Name, X-Filename
        |
        v
    Release semaphore
```

Inbound File Flow

```
Object Store watcher detects new object (UpdatesOnly mode)
    |
    v
Read metadata headers (X-Post-Id, X-Conn-Name, X-Filename)
    |
    v
Filter: does X-Conn-Name match this inbound connection?
    |
    v
Check: file passes inbound filter policy?
    |
    v
Look up local post ID via KV mapping: pm-{connName}-{remotePostID}
    |  (retries up to 3 times with linear backoff: 1s, 2s)
    v
Acquire file semaphore (max 32, blocks until slot available)
    |
    v
Download file bytes from Object Store
    |
    v
Upload file to local Mattermost (API.UploadFile)
    |
    v
Attach file to local post (append FileId via UpdatePost)
    |
    v
Release semaphore
```

File Filtering

Each connection can independently filter files by extension using three modes:

Mode	Behavior	Example
`""` (empty, default)	No filtering. All files pass.	`"file_filter_mode": ""`
`"allow"`	Only files with listed extensions are relayed.	`"file_filter_mode": "allow", "file_filter_types": ".pdf,.docx"`
`"deny"`	Files with listed extensions are blocked.	`"file_filter_mode": "deny", "file_filter_types": ".exe,.bat"`

Extensions are case-insensitive and a leading dot is optional (both `pdf` and `.pdf` are accepted). Filters are evaluated independently on outbound (before upload) and inbound (before download).

> **JetStream Requirement**
> File transfer requires a JetStream-enabled NATS server. If the NATS server does not have JetStream enabled, file transfer will fail silently with errors logged. Core NATS text relay is not affected.

XML Format

When `message_format` is set to `"xml"` on an outbound connection, messages are serialized using the XML schema defined in `schema/crossguard.xsd` (namespace: `urn:mattermost-crossguard`). The root element is `CrossguardMessage` with child elements matching the envelope fields. An XSAT report is available at `schema/crossguard-xsat-report.md` for CDS approval processes.

Key Differences from JSON

Aspect	JSON	XML
Root element	N/A (top-level object)	`<CrossguardMessage xmlns="urn:mattermost-crossguard">`

Aspect	JSON	XML
Field names	snake_case (`post_message` , `channel_name`)	PascalCase (`PostMessage` , `ChannelName`)
Message body field	`message`	`MessageText`
Character escaping	JSON string escaping	XML entity escaping (`<` , `>` , `&`)
Size overhead	Baseline	~30-50% larger (closing tags, namespace)

Inbound Auto-Detection

Inbound connections do not require format configuration. The plugin inspects the first non-whitespace byte of each incoming message: if it is `<` , the message is parsed as XML; otherwise, it is parsed as JSON. This allows a single inbound connection to accept messages from sources using either format.

> **BOM Handling**
> XML output does not include a Byte Order Mark (BOM). If a CDS device prepends a UTF-8 BOM (0xEF 0xBB 0xBF) to the message, the inbound parser will strip it automatically before format detection. BOM-prefixed XML is handled correctly.

Schema-Valid Example Payloads

The repository ships 12 schema-valid sample payloads under `schema/examples/` , one per message-type variant the plugin emits. They collectively exercise: simple posts, markdown-rich posts, fenced code blocks with literal `<` , `>` , and `&` in the body, GFM tables and reference links, threaded replies, long-form incident reports, post edits, deletes, standard-emoji reactions, custom-emoji reactions, reaction removal, and the connectivity test message. A full index lives in `schema/examples/README.md` .

Each example file is **byte-for-byte identical** to the bytes the plugin puts on the wire: the output of `model.Marshal(env, FormatXML)` plus a trailing newline, nothing else. That means a single line per message after the `<?xml?>` header (no inter-element indentation), entity-escaped `MessageText` content (`<` becomes `<` , `>` becomes `>` , `&` becomes `&` ,

embedded newlines become `
`), and no CDATA sections. The byte-equality contract is enforced by `TestExampleFiles_Symmetric` in `server/model/examples_roundtrip_test.go`, which also verifies that `Unmarshal` followed by `Marshal` is a fixed point and that marshalling is deterministic.

All sample IDs are 26 characters of `[a-z0-9]` (per the `MattermostIDType` constraint in `schema/crossguard.xsd`). Envelope `Timestamp` is ISO 8601 (RFC 3339), while `PostMessage/CreateAt` is Unix epoch **milliseconds** and is earlier than the envelope timestamp to reflect "post was created, then relayed".

To validate the examples locally:

```
for f in schema/examples/*.xml; do
  xmllint --noout --schema schema/crossguard.xsd "$f"
done
```

The simplest example, a minimal top-level post, is reproduced below. Open the file in an editor that soft-wraps, or pipe through `xmllint --format`, for a human-readable view.

Example: `01_post_simple.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<CrossguardMessage xmlns="urn:mattermost-crossguard"><Type>crossguard_post</Type><Time
```

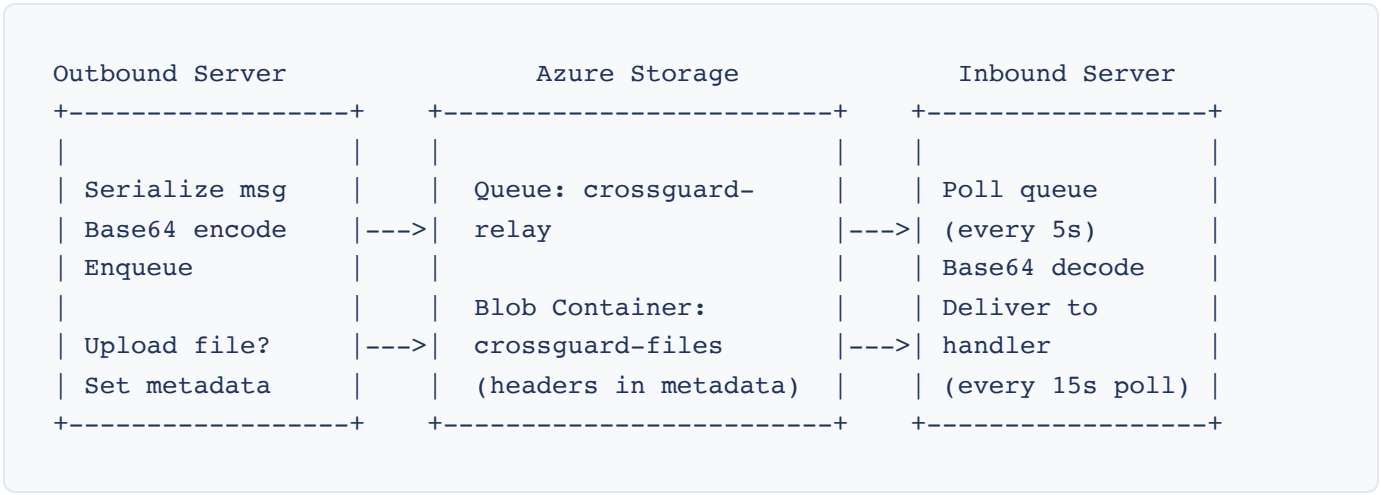
Azure Queue Storage Transport

Overview

The Azure provider uses Azure Queue Storage for message delivery and Azure Blob Storage for file transfer. Unlike NATS pub/sub, Azure uses a polling model where the inbound side periodically checks the queue for new messages.

Auth is via an Azure Storage shared-key credential (legacy default) or an Azure AD Service Principal (Phase 1 supports the client-credentials / client-secret flow). See Azure Service Principal Authentication in the admin guide for the SP field reference, the minimum RBAC roles, and the deployment guidance (including the no-auto-create rule in SP mode).

Message Flow



Queue Polling

Parameter	Value	Description
Poll Interval	5 seconds	How often the inbound side checks for new queue messages
Visibility Timeout	5 minutes	How long a dequeued message is hidden from other consumers
Batch Size	32 messages	Maximum messages dequeued per poll cycle
Message Encoding	Base64	All messages are Base64-encoded before enqueuing
Max Message Size	48,000 bytes	Safe limit before Base64 encoding to the 64 KB Azure wire limit

Message Lifecycle

1. Outbound serializes the message envelope (JSON or XML)
2. Payload is Base64-encoded and enqueued to Azure Queue Storage
3. Inbound polls the queue every 5 seconds
4. Messages are dequeued with a 5-minute visibility timeout
5. On successful processing, the message is deleted from the queue

6. On handler error, the message becomes visible again after the timeout for retry
7. Malformed messages (invalid Base64) are deleted immediately to prevent reprocessing loops

Azure File Transfer (Blob Storage)

When `file_transfer_enabled` is `true` and a `blob_container_name` is configured, file attachments are transferred via Azure Blob Storage.

Upload (Outbound)

1. File data is uploaded as a block blob to the configured container
2. The blob key follows the format: `{post_id}/{unique_id}`
3. Headers (post ID, connection name, filename) are stored as blob metadata under the `crossguard_headers` key
4. Metadata is a Base64-encoded JSON object: `{"headers": {"X-Post-Id": "...", "X-Conn-Name": "...", "X-Filename": "..."}}`

Download (Inbound)

1. The blob watcher polls the container every 15 seconds using a list operation
2. New blobs (after the last processed marker) are downloaded
3. Headers are extracted from the `crossguard_headers` metadata
4. After successful processing, the blob is deleted
5. The marker only advances after a successful delete, ensuring blobs are re-listed on failure

Azure Queue Connection Configuration

Field	JSON Key	Required	Description
Queue Service URL	`queue_service_url`	Yes	Azure Queue Storage service URL (e.g. `https://<account>.queue.core.windows.ne`
Blob Service URL	`blob_service_url`	When file transfer enabled	Azure Blob Storage service URL (e.g. `https://<account>.blob.core.windows.net`

Field	JSON Key	Required	Description
Account Name	`account_name`	Yes	Azure Storage account name
Account Key	`account_key`	Yes	Azure Storage shared access key (base64, trea secret)
Queue Name	`queue_name`	Yes	Azure Queue name (auto-created if missing)
Blob Container Name	`blob_container_name`	When file transfer enabled	Azure Blob container (auto-created if missing)

Example: Azure Queue Connection

```
{
  "name": "azure-relay",
  "provider": "azure-queue",
  "file_transfer_enabled": true,
  "azure_queue": {
    "queue_service_url": "https://mystorageaccount.queue.core.windows.net",
    "blob_service_url": "https://mystorageaccount.blob.core.windows.net",
    "account_name": "mystorageaccount",
    "account_key": "base64-account-key",
    "queue_name": "crossguard-relay",
    "blob_container_name": "crossguard-files"
  }
}
```

Auto-Provisioning

Unlike NATS, the Azure Queue provider automatically creates both the queue and the blob container on startup if they do not already exist. The "already exists" response (HTTP 409) is treated as success. No manual Azure infrastructure setup is required beyond creating the storage account and providing credentials.

Azure Blob Storage Transport

Overview

The Azure Blob provider is a second Azure transport that does not use Azure Queue Storage at all. Instead of enqueuing one queue message per relay event, the outbound side appends messages to an in-memory write-ahead log (WAL) and periodically seals and uploads the WAL as a single batch blob. The inbound side polls the blob container, takes a short-lived lease on each batch blob, processes its messages, and deletes the blob. File attachments are uploaded as deferred file blobs into the same container.

Use this transport when you want to amortize blob operations across many messages, avoid per-message queue charges, or when the queue service is unavailable in your environment. Expected end-to-end latency is bounded by `flush_interval_seconds`.

Auth is via an Azure Storage shared-key credential (legacy default) or an Azure AD Service Principal. See Azure Service Principal Authentication in the admin guide for the SP field reference, the minimum RBAC role (`Storage Blob Data Contributor`), and the no-auto-create rule in SP mode.

Message Lifecycle

1. Outbound appends each relay event to an in-memory WAL buffer.
2. Every `flush_interval_seconds` (default 60), the buffer is sealed and uploaded as a batch blob under a time-ordered key.
3. Inbound lists the container on a poll cycle and, for each batch blob, acquires a lease.
4. While the lease is held, messages are delivered to the inbound handler.
5. On success, the blob is deleted and the lease is released. On failure, the lease expires and another poller may retry.
6. If a lease grows older than `blob_lock_max_age_seconds` (default 300), it is considered stale and another inbound node may break the lease and take over.

Azure Blob Connection Configuration

Field	JSON Key	Required	Description
Service URL	`service_url`	Yes	Azure Blob Storage service URL
Account Name	`account_name`	Yes	Azure Storage account name
Account Key	`account_key`	Yes	Azure Storage shared access key (base64, treat as secret)
Blob Container Name	`blob_container_name`	Yes	Container holding both batch WAL blobs and deferred file blobs (auto-created if missing)
Flush Interval	`flush_interval_seconds`	No	Seconds between WAL flushes. Default 60, minimum 5.
Blob Lock Max Age	`blob_lock_max_age_seconds`	No	Seconds before a stale lease may be broken. Default 300, minimum 30.

Example: Azure Blob Connection

```
{
  "name": "azure-blob-relay",
  "provider": "azure-blob",
  "file_transfer_enabled": true,
  "azure_blob": {
    "service_url": "https://mystorageaccount.blob.core.windows.net",
    "account_name": "mystorageaccount",
    "account_key": "base64-account-key",
    "blob_container_name": "crossguard-wal",
    "flush_interval_seconds": 60,
    "blob_lock_max_age_seconds": 300
  }
}
```

Azure Service Bus Transport

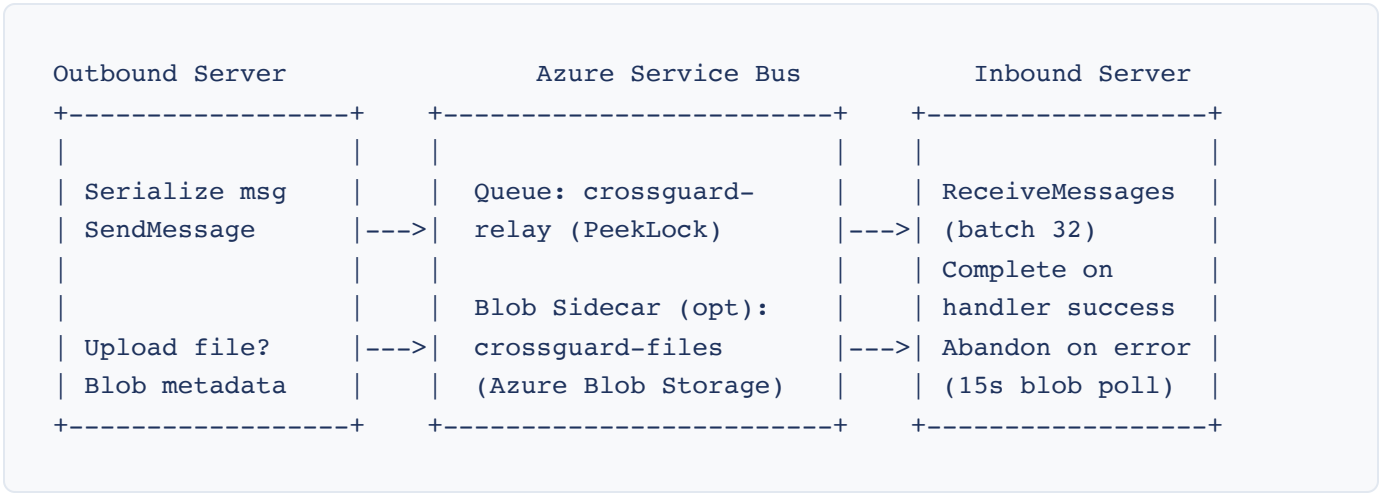
Overview

The Azure Service Bus provider uses an AMQP-based Service Bus queue for message delivery and an optional Azure Blob Storage sidecar container for file transfer. Unlike the Azure Queue provider, Service Bus uses PeekLock receive semantics with server-side delivery counts and a native dead-letter queue (DLQ). Authentication is via a SAS connection string (legacy default) or an Azure AD Service Principal. In connection-string mode the blob sidecar takes its own shared-key credential separate from the Service Bus namespace; in Service Principal mode the blob sidecar inherits the parent SP credential so a single AAD identity covers both AMQP and blob access.

See Azure Service Principal Authentication in the admin guide for the SP field reference (including the additional `service_bus_namespace` required in SP mode), the minimum RBAC roles (`Azure Service Bus Data Sender` + `Azure Service Bus Data Receiver` on the queue, plus `Storage Blob Data Contributor` on the blob account when file transfer is enabled), and the credential-inheritance rule for the blob sidecar.

The plugin does *not* auto-create queues. Pre-create the queue in the Azure portal or via ARM/Terraform with your chosen `LockDuration` and `MaxDeliveryCount` before saving the connection. Lock duration is an entity property; the plugin cannot set it.

Message Flow



Receive Parameters

Parameter	Value	Description
Receive Mode	PeekLock	Messages are locked on receive; the plugin calls Complete on success or Abandon on handler error.
Receive Batch Size	32 messages	Maximum messages requested per receive call.
Receive Max Wait	5 seconds	Per-call context deadline. Empty-queue waits are bounded by this timeout, then the loop restarts.
Ack Timeout	10 seconds	Detached timeout for Complete / Abandon / Close so shutdown races cannot starve settlement.
Blob Poll Interval	15 seconds (configurable)	How often the inbound side polls the blob sidecar container for file attachments.
Default Max Message Size	192,000 bytes	Default ceiling for outbound payload size (Service Bus Standard tier caps messages at 256 KiB). Configurable up to 100 MiB on Premium.

Message Lifecycle

1. Outbound serializes the message envelope (JSON or XML) and sends it via the Service Bus SDK.
2. Oversized payloads (above `max_message_size_bytes`) are rejected at the client boundary before send.
3. Inbound receives messages in batches (up to 32) under PeekLock.
4. Messages with `DeliveryCount > 1` log a WARN (error code `26004`) so operators can detect poison-message buildup before DLQ moves happen.
5. Empty-body messages are dropped via Complete to avoid infinite redelivery loops (error code `26005`).
6. On handler success, the message is Completed. On handler error, the message is Abandoned; Service Bus increments `DeliveryCount` and redelivers.
7. After `MaxDeliveryCount` is exceeded, Service Bus auto-moves the message to the queue's dead-letter sub-queue. The plugin does NOT inspect or process DLQ messages; use the Azure portal, `az servicebus queue message`, or Azure monitoring to inspect the DLQ.

Service Bus File Transfer (Blob Sidecar)

When `file_transfer_enabled` is `true` and the blob sidecar fields are configured, file attachments are transferred via Azure Blob Storage using the same upload/download mechanics as the Azure Queue provider (`{post_id}/{unique_id}` blob keys, Base64-encoded JSON metadata under the `crossguard_headers` key). Service Bus queues themselves are message-only; files never flow through the queue.

Service Bus Connection Configuration

Field	JSON Key	Required	Description
Connection String	`connection_string`	Yes	SAS connection string for the Service Bus namespace. Treat as a secret.
Queue Name	`queue_name`	Yes	Pre-created queue name or hierarchical path. Up to 260 characters; letters, digits, periods, hyphens, underscores, and forward slashes.
Blob Service URL	`blob_service_url`	Only when `file_transfer_enabled` is true	Azure Blob Storage endpoint for the file sidecar.
Blob Account Name	`blob_account_name`	Only when `file_transfer_enabled` is true	Storage account name for the blob sidecar (independent of

Field	JSON Key	Required	Description
			the Service Bus namespace).
Blob Account Key	`blob_account_key`	Only when `file_transfer_enabled` is true	Shared access key for the blob sidecar storage account. Treat as a secret.
Blob Container Name	`blob_container_name`	Only when `file_transfer_enabled` is true	Blob container used for file attachments. Auto-created on startup if it does not exist.
Max Message Size	`max_message_size_bytes`	No	Outbound payload ceiling. Default 192000. Range 1024 to 104857600 (100 MiB Premium cap).
Blob Poll Interval	`blob_poll_interval_seconds`	No	Inbound blob-sidecar poll cadence. Default 15, minimum 1.

Example: Azure Service Bus Connection

```
{
  "name": "azure-servicebus-relay",
  "provider": "azure-servicebus",
  "file_transfer_enabled": true,
  "azure_servicebus": {
    "connection_string": "Endpoint=sb://myns.servicebus.windows.net/;SharedAccessKeyNa
    "queue_name": "crossguard-relay",
    "blob_service_url": "https://mystorageaccount.blob.core.windows.net",
    "blob_account_name": "mystorageaccount",
```

```
    "blob_account_key": "base64-account-key",
    "blob_container_name": "crossguard-files"
  }
}
```

Loop Prevention

Cross Guard prevents infinite relay loops through multiple mechanisms:

- **Relayed property:** Every post created by inbound message processing is tagged with a `crossguard_relayed` post property. Outbound hooks skip posts with this property.
- **System message filtering:** System-generated messages (join/leave, header changes, etc.) are never relayed.
- **Bot post filtering:** Posts from the relay bot itself are excluded from outbound processing.
- **Connection linkage validation:** Messages are only published to outbound connections that are explicitly linked to the originating team. Inbound messages are only processed if the target team and channel are linked to the receiving inbound connection.

> **OpenAPI Specification**
> The machine-readable OpenAPI 3.1 specification for this interface is available at `schema/crossguard-api.yaml` in the plugin source repository.